



# Chapter 10 — Security, Deployment & DevOps

## 10.1 Security Model

### Authentication

- **JWT-based auth:** short-lived access tokens (15 min) + rotating refresh tokens (7 days, single-use rotation with reuse detection).
- Passwords hashed with **argon2id**; login rate-limited to 5 attempts/min per IP.
- WebSocket connections authenticate via the same JWT at handshake; expired tokens force a clean re-auth + resume cycle.

### Role-Based Access Control (RBAC)

| Capability                        | viewer | operator | planner | admin |
|-----------------------------------|--------|----------|---------|-------|
| View dashboards, map, feeds       | ✓      | ✓        | ✓       | ✓     |
| Acknowledge/resolve incidents     | ✗      | ✓        | ✓       | ✓     |
| Use AI chat / RAG                 | ✗      | ✓        | ✓       | ✓     |
| Create/optimize/launch missions   | ✗      | ✗        | ✓       | ✓     |
| Start/stop/pause runs, set speed  | ✗      | ✗        | ✓       | ✓     |
| Author scenarios, ingest RAG docs | ✗      | ✗        | ✗       | ✓     |
| Manage users, view audit logs     | ✗      | ✗        | ✗       | ✓     |

Enforced at three layers: route dependency (FastAPI), WebSocket topic authorization, and UI affordance hiding (defense in depth — the API is the real gate).

### Audit logging

- Every mutating action (login, run control, incident actions, mission launch, scenario upload, user management) writes to `audit_logs` with user, action, target,

and detail.

- AI outputs are audited too: which model version produced which sitrep from which events — full provenance.

## Transport & application security

- TLS everywhere via Nginx (self-signed for dev, Let's Encrypt for public demo).
- Security headers: HSTS, CSP, X-Content-Type-Options, frame-ancestors none.
- CORS locked to the frontend origin; cookies (if used) are `Secure` + `HttpOnly` + `SameSite=Strict`.
- Rate limiting (Redis token bucket): 100 req/min default, 10/min AI endpoints, 5/min auth.
- Input hygiene: Pydantic validation on every request; parameterized SQL only; scenario YAML parsed with a safe loader and whitelisted action/condition DSL; uploaded videos type/size-checked and stored outside the web root in MinIO.
- Secrets via environment variables / K8s secrets — never in the repo; `.env.example` documents every variable.
- Dependency scanning: Dependabot + `pip-audit` / `npm audit` in CI.

## 10.2 Deployment Topologies

### Development & Phases 1–6 — Docker Compose

```
# docker-compose.yml (summary)
services:
  nginx:      # reverse proxy, TLS, static frontend
  frontend:  # Vite build → static assets
  api:        # FastAPI monolith (gateway + domains + simulation)
  ai:         # AI service (LangGraph, RAG)
  ollama:     # local LLM runtime (volume for model weights)
  vision:     # Celery worker (Phase 4+)
  analytics:  # forecasting service (Phase 6+)
  postgres:   # postgres/postgis:16 + pgvector
  redis:      # pub/sub, cache, rate limiting
  minio:      # object storage
  prometheus: # metrics (Phase 7, optional earlier)
  grafana:    # dashboards
```

One command — `docker compose up` — brings up the full platform. `make seed` loads the demo scenario.

## Phase 7 — Kubernetes (k3s single node)

```
Namespace: doip
├─ Deployments: api (2), ai (1), vision-worker (1..3, HPA on queue depth),
│  analytics (1), frontend (nginx, 2)
├─ StatefulSets: postgres, kafka, minio
├─ Services + Ingress (TLS): app.doip.local → frontend, api.doip.local → api
├─ ConfigMaps: app config, scenario library
├─ Secrets: DB creds, JWT signing key, MinIO keys
└─ Monitoring: kube-prometheus-stack (Prometheus, Grafana, Alert manager)
```

Deployment via a single **Helm chart** with `values-dev.yaml` / `values-demo.yaml`. Rollouts are rolling with readiness probes; the simulation engine runs as a leader-elected single active instance to preserve world-state authority (ADR-6).

## 10.3 CI/CD Pipeline (GitHub Actions)

```
on: [push, pull_request]

1. lint-and-type: Ruff + mypy (backend) · ESLint + tsc (frontend)
2. test:          pytest (unit + integration, Postgres/Redis services)
                  · vitest (frontend) · golden-run determinism suite
3. e2e (main only): Playwright against docker-compose stack
4. eval-ai (nightly): sitrep grounding + RAG golden set + CV validation clip
5. build:         docker buildx bake → tag with git SHA → push to GHCR
6. deploy (tags): helm upgrade --install → smoke test → notify
```

**Branch policy:** trunk-based; short-lived feature branches; PRs require green CI. Release tags `v<phase>.<sprint>` map to the roadmap (e.g., `v3.2` = Phase 3, Sprint 2).

## 10.4 Observability

### Metrics (Prometheus)

| Metric                                                          | Type      | Why it matters                         |
|-----------------------------------------------------------------|-----------|----------------------------------------|
| <code>sim_tick_duration_seconds</code>                          | histogram | Engine health; must stay < 50 ms (NFR) |
| <code>sim_entities_active</code>                                | gauge     | Load context for every other metric    |
| <code>events_published_total</code> (by type)                   | counter   | Throughput; detects silent stalls      |
| <code>ws_clients_connected</code> / <code>ws_lag_seconds</code> | gauge     | Real-time delivery health (NFR-1)      |
| <code>ai_request_duration_seconds</code> (by pipeline)          | histogram | Sitrep/RAG latency budgets             |
| <code>vision_job_queue_depth</code>                             | gauge     | Drives the HPA in K8s                  |
| <code>http_requests_total</code> / error rate                   | counter   | Standard RED metrics                   |

### Dashboards & alerts (Grafana)

- **Platform Ops:** tick duration, event lag, WS clients, API error rate.
- **AI Ops:** pipeline latency, grounding-check failures, eval scores over time.
- **Alert rules:** tick > 100 ms for 1 min; WS lag > 2 s; event bus consumer lag; disk > 80%; error rate > 2%.

### Logging

- Structured JSON logs everywhere ( `structlog` ), correlated by `request_id` / `run_id` / `event_id` .
- Loki (optional) aggregates logs next to metrics; a `run_id` filter reconstructs the story of any simulation.

## 10.5 Backup & Recovery

- Nightly `pg_dump` + weekly base backup; MinIO bucket versioning for models/footage.
- Completed runs exportable as a single archive (events + scenario + seed) — restorable and replayable on any instance.
- Recovery drill documented: restore DB, restart stack, resume from snapshot — rehearsed once per phase.

## 10.6 Environments

| Env               | Purpose           | Stack                                                                     |
|-------------------|-------------------|---------------------------------------------------------------------------|
| <code>dev</code>  | Daily development | Compose, hot reload, seeded demo scenario                                 |
| <code>test</code> | CI                | Ephemeral Compose services, deterministic seeds                           |
| <code>demo</code> | Public showcase   | k3s (Phase 7) or Compose on a VPS, TLS, seeded scenarios, viewer accounts |